1D DP - Lecture 2

04 November 2025 22:44

* Always Remember:→ Try Your Best to come up with the Recursive logic only, Because after that DP is just 5 minutes of work. — vHizen

* Fibonacci Sequence → 0, 1, 1, 2, 3, 5, 8, 13

Any number is sum of previous two values

Ques → find n^{th} fibonacci

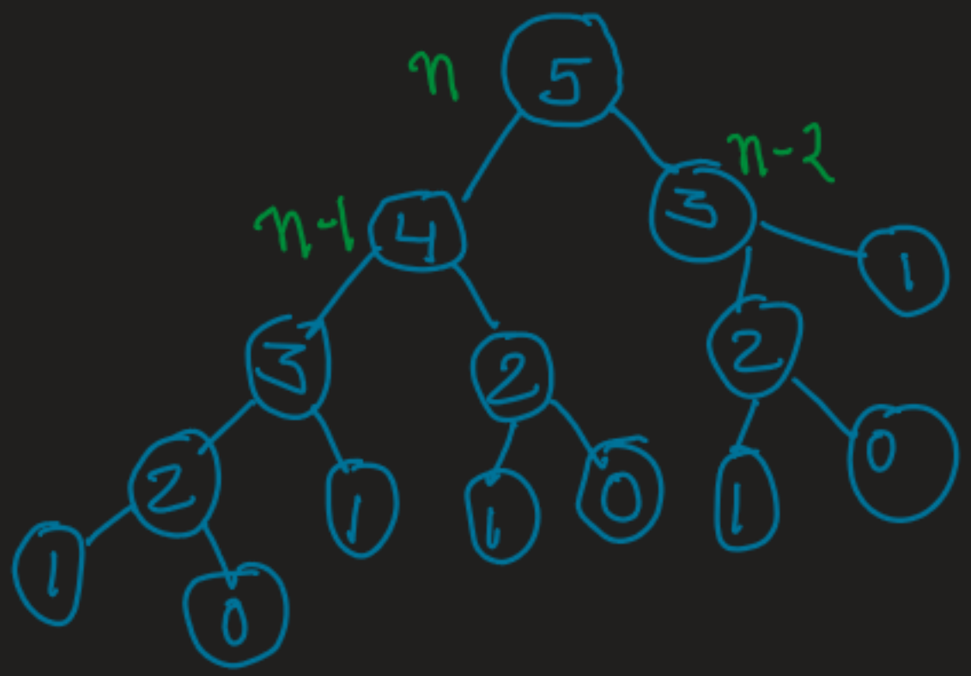
ex:- 3^{th} fib. number → 2

5^{th} —|— → 5

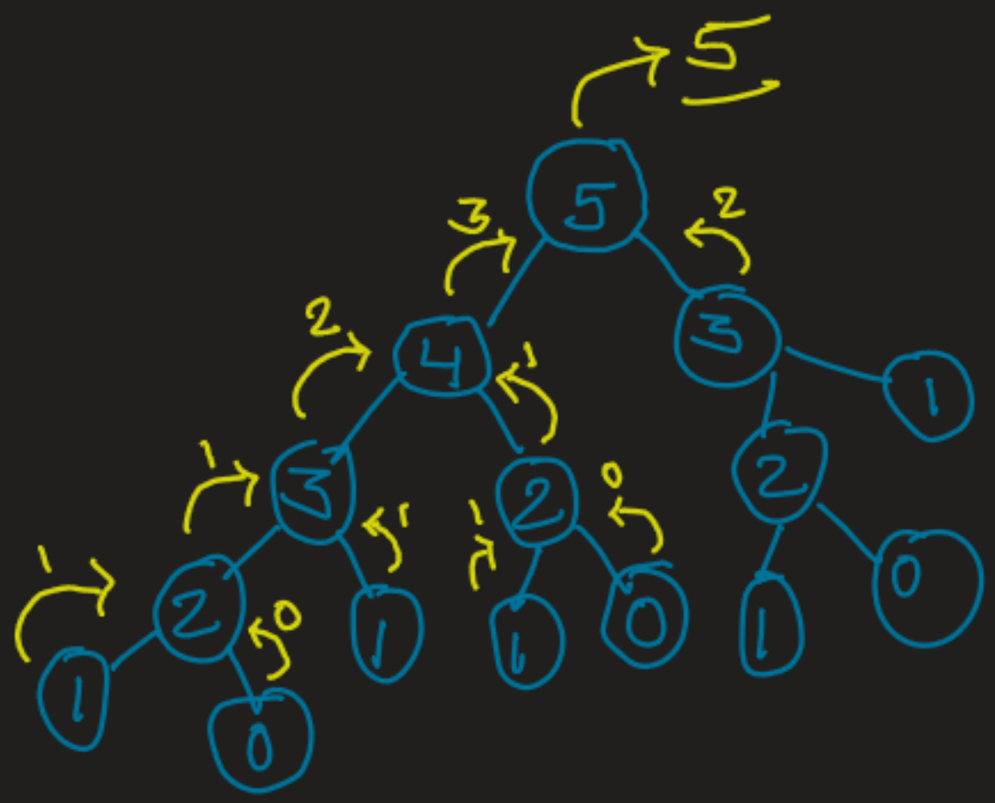
6^{th} —|— → 8

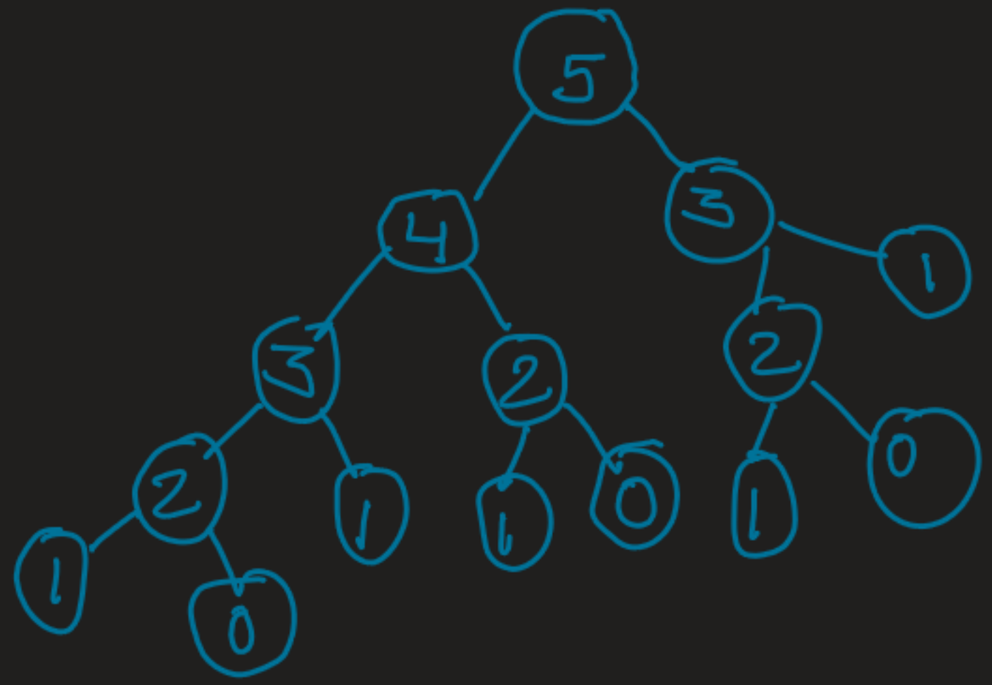
* Tree Diagram :-

$n = 5$



→ solving small problems (subproblems) to solve a Big Problem.

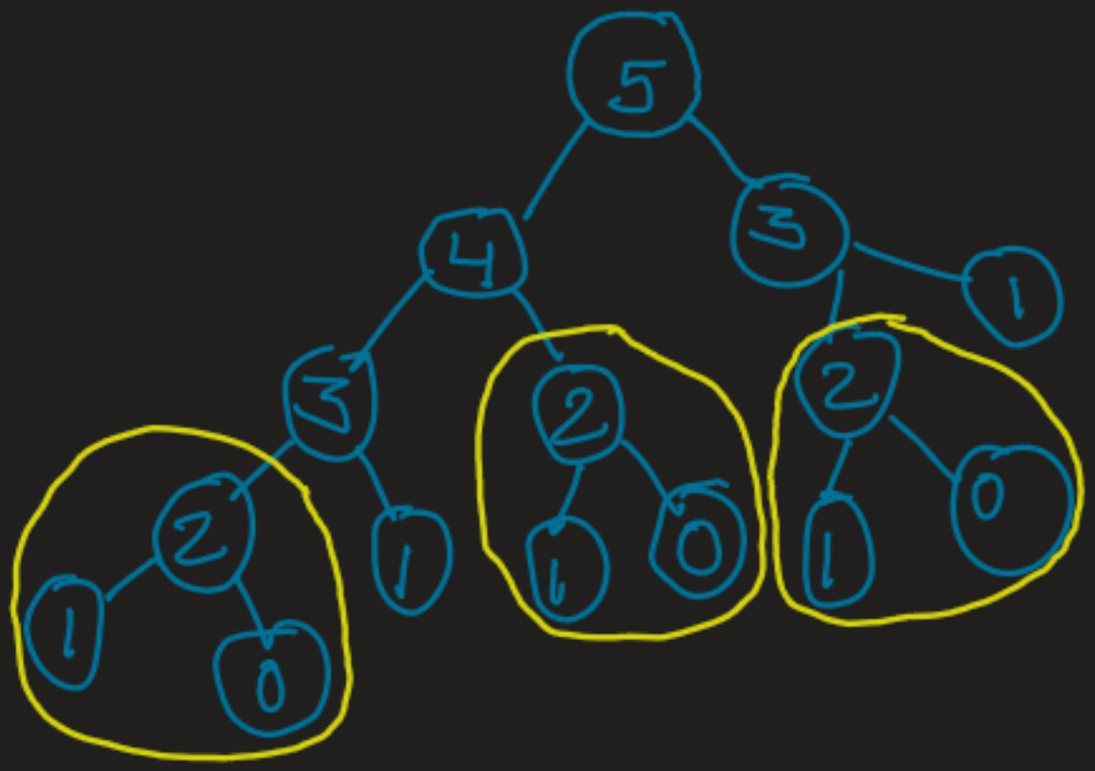




* Time → $O(2^N)$, $n=5$ | 5, 4, 3, 2, 1

→ For any value we're exploring two options / possibilities.

: Why It's taking Exponential time?



→ Overlapping Sub problems (Repitative work)

→ Means when n is increases, the amount of sub-problem increases

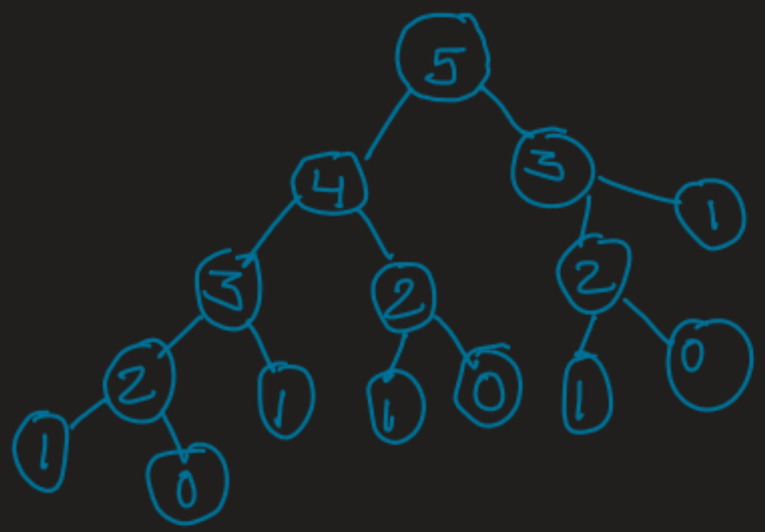
→ Solving this repetative problems again and again is increasing time.

→ This concludes that we could optime time in our recursion.

→ Now this hints to DP.

→ So for recursion, we apply Top-Down DP,
So lets do Memoization.

* let's identify Base cases :-



→ if (n == 1) return 1

→ if (n == 0) return 0

* For rest part :-

(we just need previous two values) :

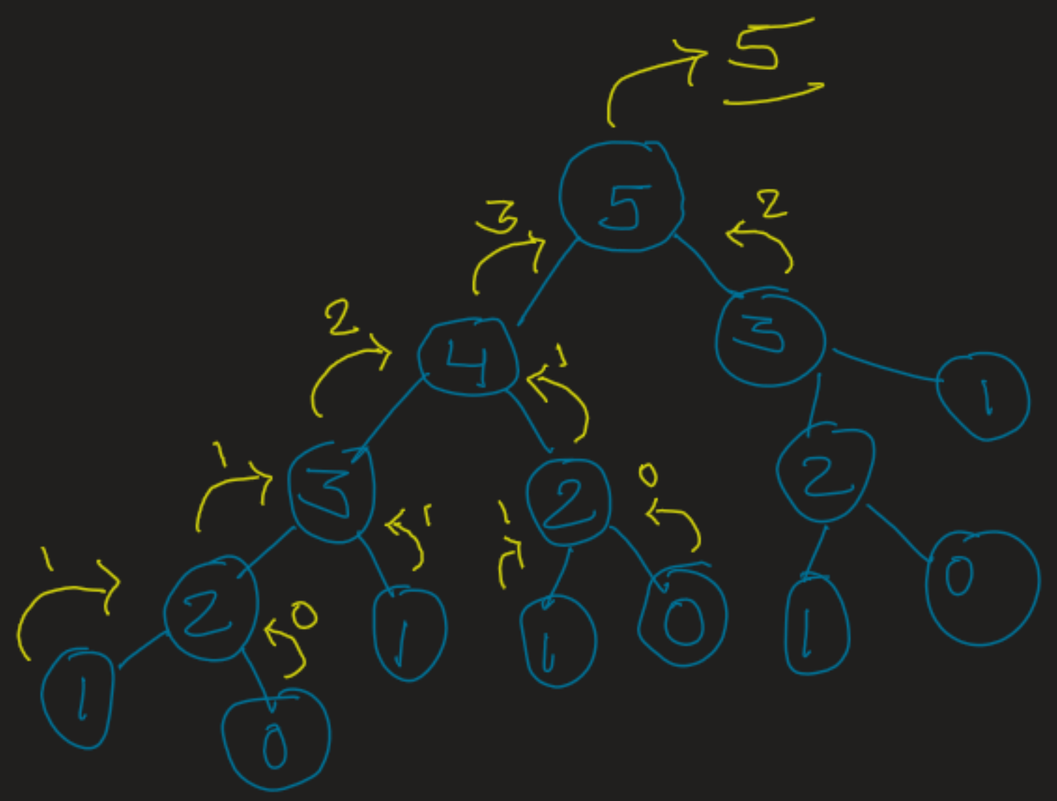
→ int prevVal1 (find recursively)

int prevVal2 (find recursively)

→ return sum of both values

~ Recursive code:-

```
int nthFib(int n){
    if(n==0) return 0;
    if(n==1) return 1;
    int prevVal1 = nthFib(n-1);
    int prevVal2 = nthFib(n-2);
    return prevVal1 + prevVal2;
}
```



~> Now memoize it (Apply DP)

~> let's code it